

A.1 Foundations, Gradient descent, and backpropagation



Tiam Heydari
Postdoctoral Fellow, UBC



Welcome To Modern AI Summer Workshop!

Tiam Heydari
Chenkai Li

MODERN AI SUMMER INTENSIVE

A five-day hands-on workshop intended to
build foundational understanding and skills.

July 6 - 10, 2026
1:30 pm - 6:30 pm
Gordon B. Shrum Building



THE UNIVERSITY OF BRITISH COLUMBIA
School of Biomedical Engineering
Faculty of Applied Science | Faculty of Medicine



Time (PM)	Day 1 Monday, July 6th	Day 2 Tuesday, July 7th	Day 3 Wednesday, July 8th	Day 4 Thursday, July 9th	Day 5 Friday, July 10th
12:00-1:00					Keynote Speaker Jason Tyler Rolfe Co-founder, CTO, Variational AI
1:00-1:30					Coffee Break
1:30-2:15	Lecture A.1 Foundations, Gradient descent, and backpropagation	Lecture B.4 Graph-structured data: GNNs	Lecture D.2 Representation Learning: training and evaluation	Lecture E.3 Generative modeling: diffusion models I	Lecture F.1 Introduction to Multimodal Representation & Alignment
2:15-2:30	Coffee Break	Coffee Break	Coffee Break	Coffee Break	Coffee Break
2:30-3:15	Lecture B.1 Tabular data: MLPs	Lecture C.1 Attention and Transformer architectures	Guest Speaker Steven ten Holder Co-Founder & CEO Zeroshot Biolabs	Guest Speaker Carl de Boer Associate Professor UBC	Lecture F.2 Multimodal Fusion
3:15-3:30	Coffee Break	Coffee Break	Coffee Break	Coffee Break	Coffee Break
3:30-4:00	Tutorial 1: Basics of Pytorch	Tutorial 3: CNNs	Tutorial 5: Contrastive learning	Tutorial 7: VAEs	Tutorial 9: Multimodal
4:00-4:15	Coffee Break	Coffee Break	Coffee Break	Coffee Break	Coffee Break
4:15-5:00	Lecture B.2 Grid-structured data: CNNs	Lecture C.2 Transformer: Application and training	Lecture E.1 Generative modeling: concepts	Lecture E.4 Generative modeling: diffusion models II	Lecture F.3 Cross modal Learning
5:00-5:15	Coffee Break	Coffee Break	Coffee Break	Coffee Break	Closing remarks
5:15-6:00	Lecture B.3 Sequential data: sequence models	Lecture D.1 Representation Learning: concepts	Lecture E.2 Generative modeling: VAE	Lecture E.5 Generative modeling: Structured (conditional) predictions	
6:00-6:30	Tutorial 2: Basics of Pytorch	Tutorial 4: Attention	Tutorial 6: Evaluating Representations	Tutorial 8: Diffusion model	



Welcome To Modern AI Summer Workshop!

Why we made this workshop?

- **AI is accelerating:** it's reshaping engineering, biology, and cognition fast.
- **It's genuinely exciting:** On it's own, modern AI is creative, beautiful, and it can change how you think about problems.
- **For the future AI scientists:** If you want to work *in* AI, you need some depth and coverage, we hope to provide both up to certain degree.
- **For the researchers who'll use it:** if you want to apply AI in your own work, understanding *how* it works matters: knowing what AI can do with your data can change how you design your experiments.



Welcome To Modern AI Summer Workshop!

Our three aims:

- **The big picture:** The main paradigms of AI, and how they connect.
- **Enough depth:** To be able to read and understand recent papers using these approaches.
- **Practical:** get to where you can implement any topic we cover, by yourself, on your own data or provided data.



Welcome To Modern AI Summer Workshop!

Structure of the workshop

- **Modules:** Foundations (A, B, C, D) $\rightarrow$ advanced (E, F)
- **Recipe cards:** Step-by-step application of a complex, real ML task, end to end.
- **Tutorials (hands-on):** Implement the core concepts yourself, in practice, on your own data or provided data.
- **Guest lectures:** Applications of AI in cutting-edge research and industry, each linked to the corresponding module.



Welcome To Modern AI Summer Workshop!

What to expect

- **Concepts, visualizations and equations:** We will show the math, but always alongside applications, concepts, visuals, and examples.
- **Something for everyone:** Some of you might find certain parts easy, others more challenging, depending on your background and prior exposure to math, AI or even biology.
- **We hope everyone takes something out of it:** Whatever your starting point.
- **We will pause for questions at regular intervals,** But we have a lot of materials to cover!



Welcome To Modern AI Summer Workshop!

Sources & inspiration courses

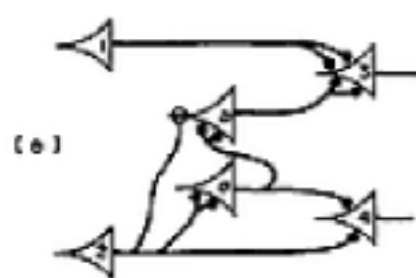
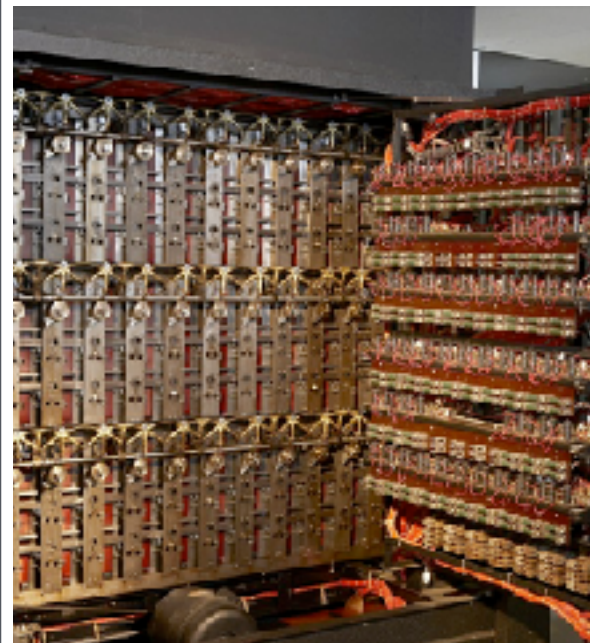
- **Deep Learning:** MIT 6.7960 · Phillip Isola
- **ML with Graphs:** Stanford CS224W · Jure Leskovec
- **Multi-Task & Meta Learning:** Stanford CS330 · Chelsea Finn
- **Deep Generative Models:** Stanford CS236 · Stefano Ermon
- **Flow Matching & Diffusion:** MIT · Peter & Ezra Erives
- **Multimodal ML:** CMU 11-777 · Morency & Liang
- **ML for Bioinformatics:** Sharif University of Technology. Ali Sharifi-Zarchi (in Persian)

+ *books and papers cited throughout the modules*

Introduction: **A Brief History of AI**



*Gauss / Legendre:
learning parameters
from data(least squares)*



*Ancient Myth:
Talos, giant
automaton made
of bronze*

*Aristotle:
Formalizing
Logic, reasoning,
thoughts given by
rule*

*Descartes: Asked
whether a mind is
something a
machine could
ever have*

*Boole/Turing:
reduced logic to
binary and defined
computation*

*McCulloch/Pitts:
first
mathematical
model of a
neuron*

*Rosenblatt:
Perceptron, a
machine that
learn*

*Hinton/
Linnainmaa/
Werbos/
Rumelhart/
Williams:
Backpropagation,*

*Hinton/
Krizhevsky/
Sutskever :
Deep learning
via Alexnet,*

*Google
Research:
Transformers,
LLMs*

~700BCE
and before

~350BCE

~1637

1805

~ 1900-1940

~ 1943

~ 1958

~ 1986

~ 2012

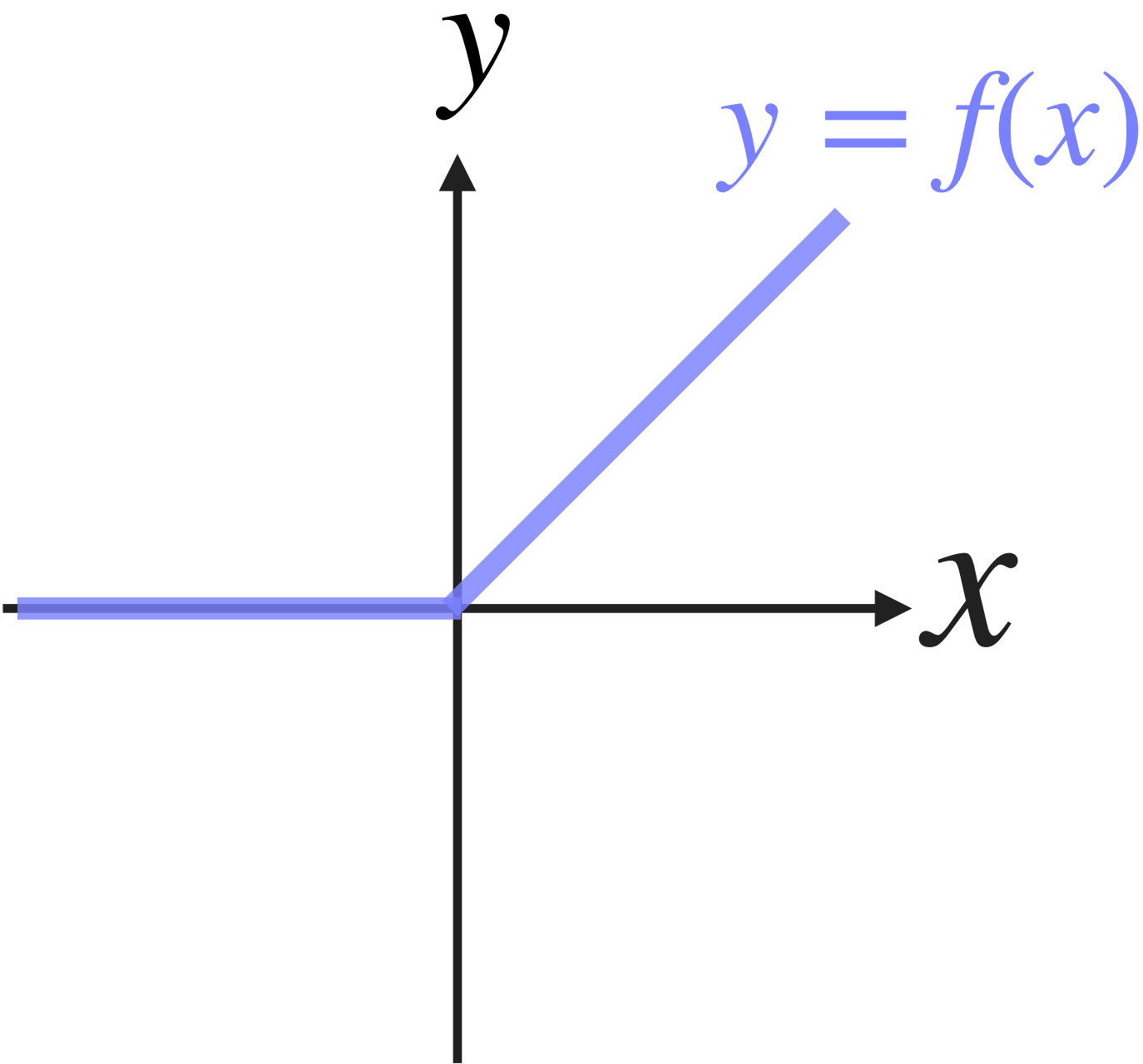
~ 2017

Introduction: Gradient descent and backpropagation

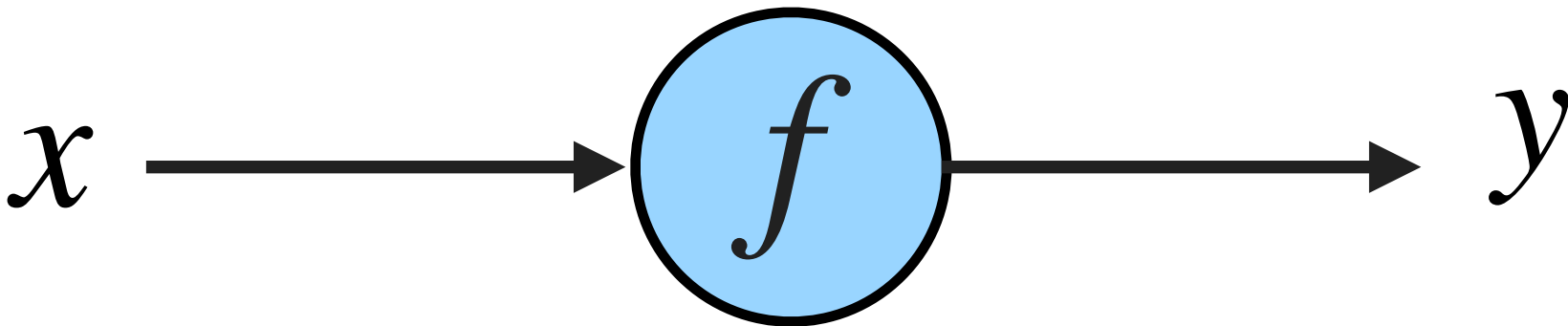
Introduction: As simple non linear function

First, consider a simple mathematical function f .
It transforms an input x into an output y :

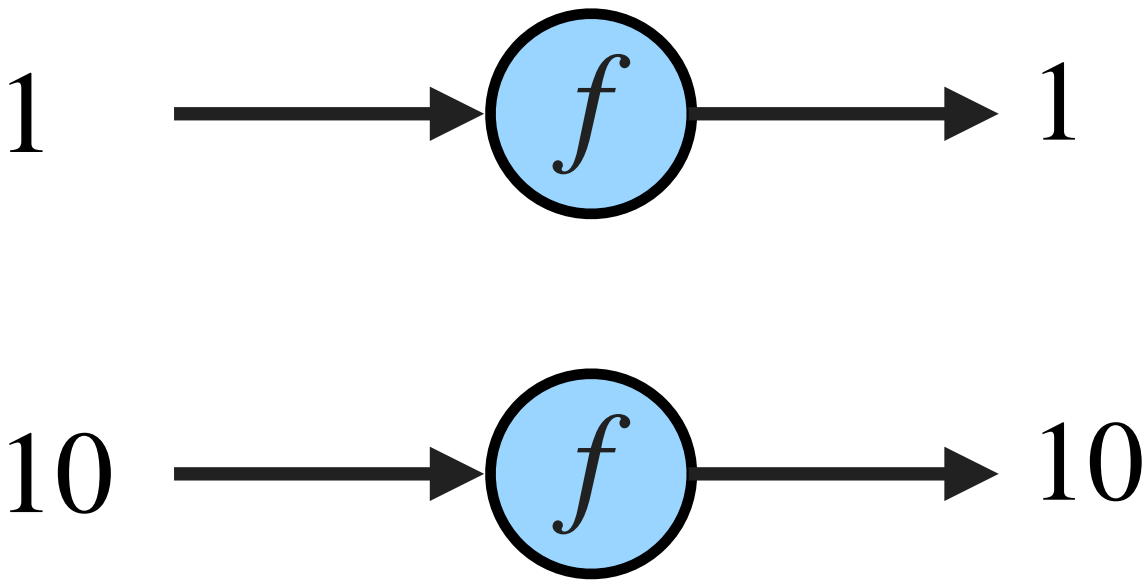
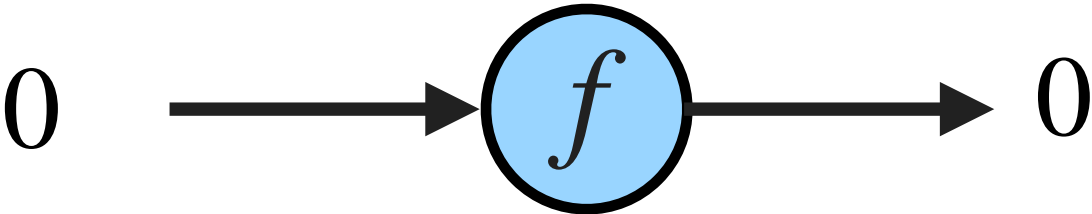
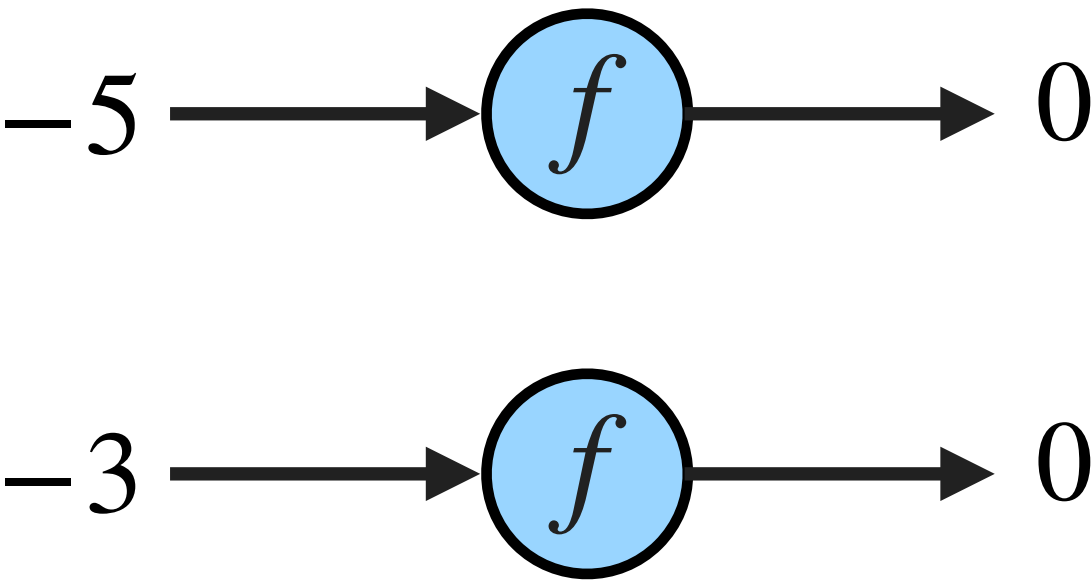
$$y = f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



Visually we can show this with the diagram bellow:

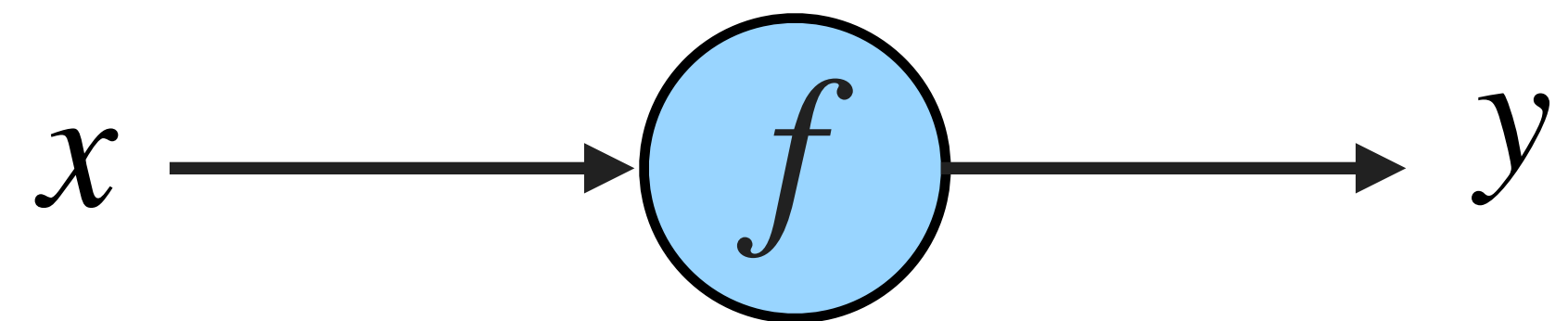


The graph describes how inputs are transformed into outputs. For example:



Introduction: Computational graph

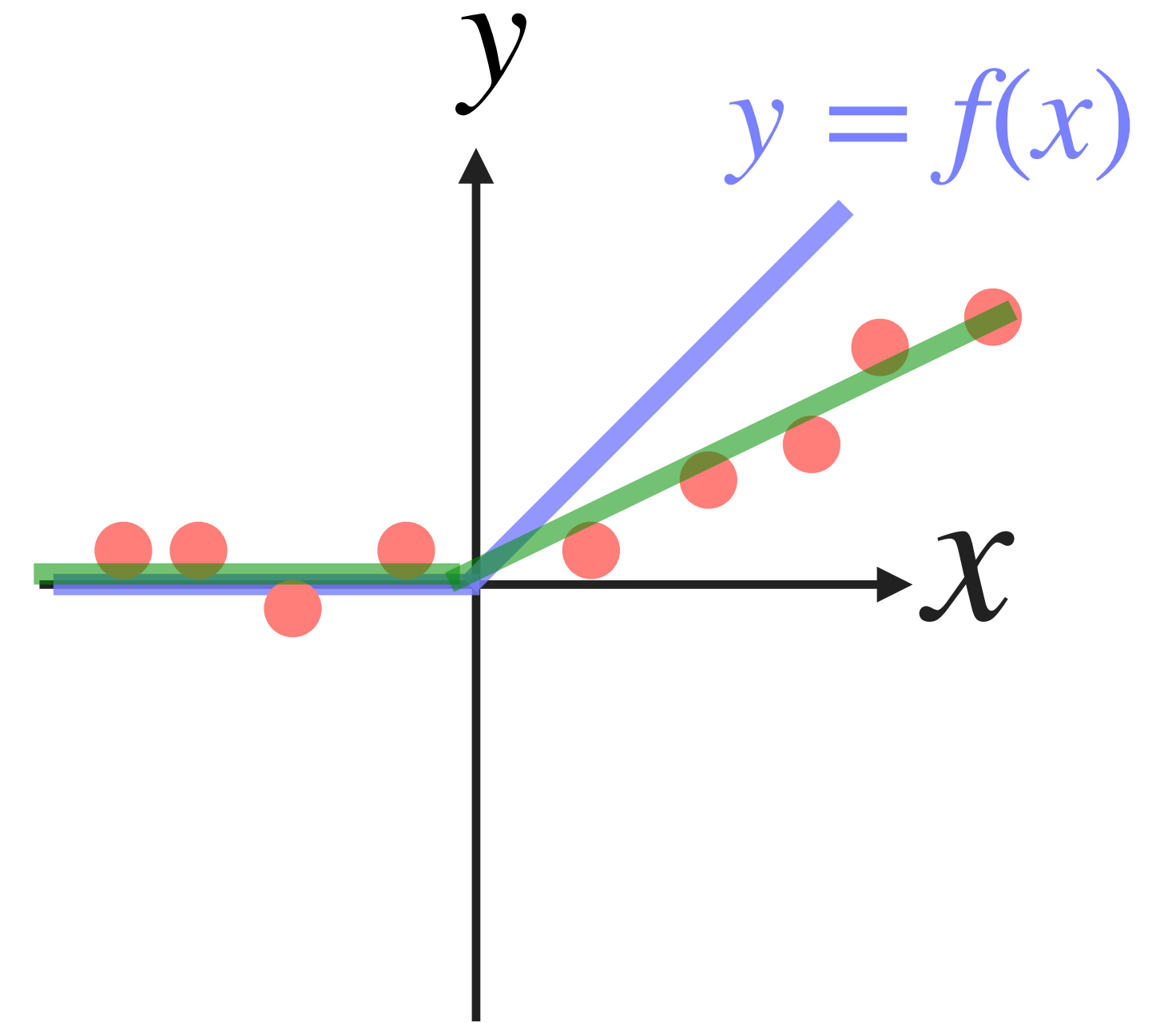
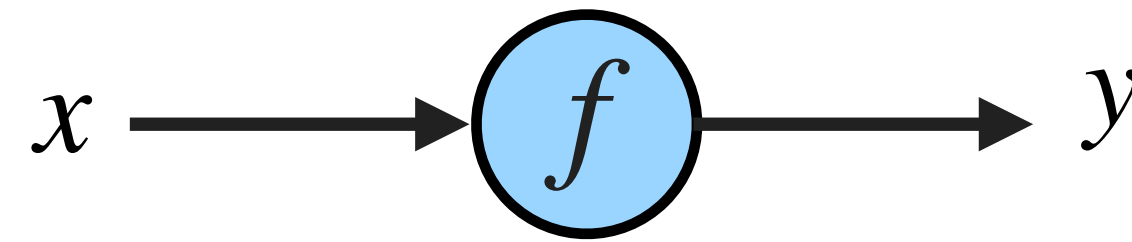
We call this type of drawings, **the computational graph**



Introduction: As simple non-linear function a predictor

Now suppose we choose the function f to make predictions.

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



We are given a dataset of input-output pairs:

$$\text{Data} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

Question: Is this function a good predictor for our data?

Not quite.

For $x < 0$, the function predicts values close to the data.

But for $x > 0$, the blue line is too steep.

But this parametrized function is a better predictor when $w = \frac{1}{2}$

$$f_w(x) = \begin{cases} 0 & \text{if } wx < 0 \\ wx & \text{if } wx \geq 0 \end{cases}$$

Introduction: How do we choose the parameter w ?

What if we do not know the best value of w ? Can we find it computationally?

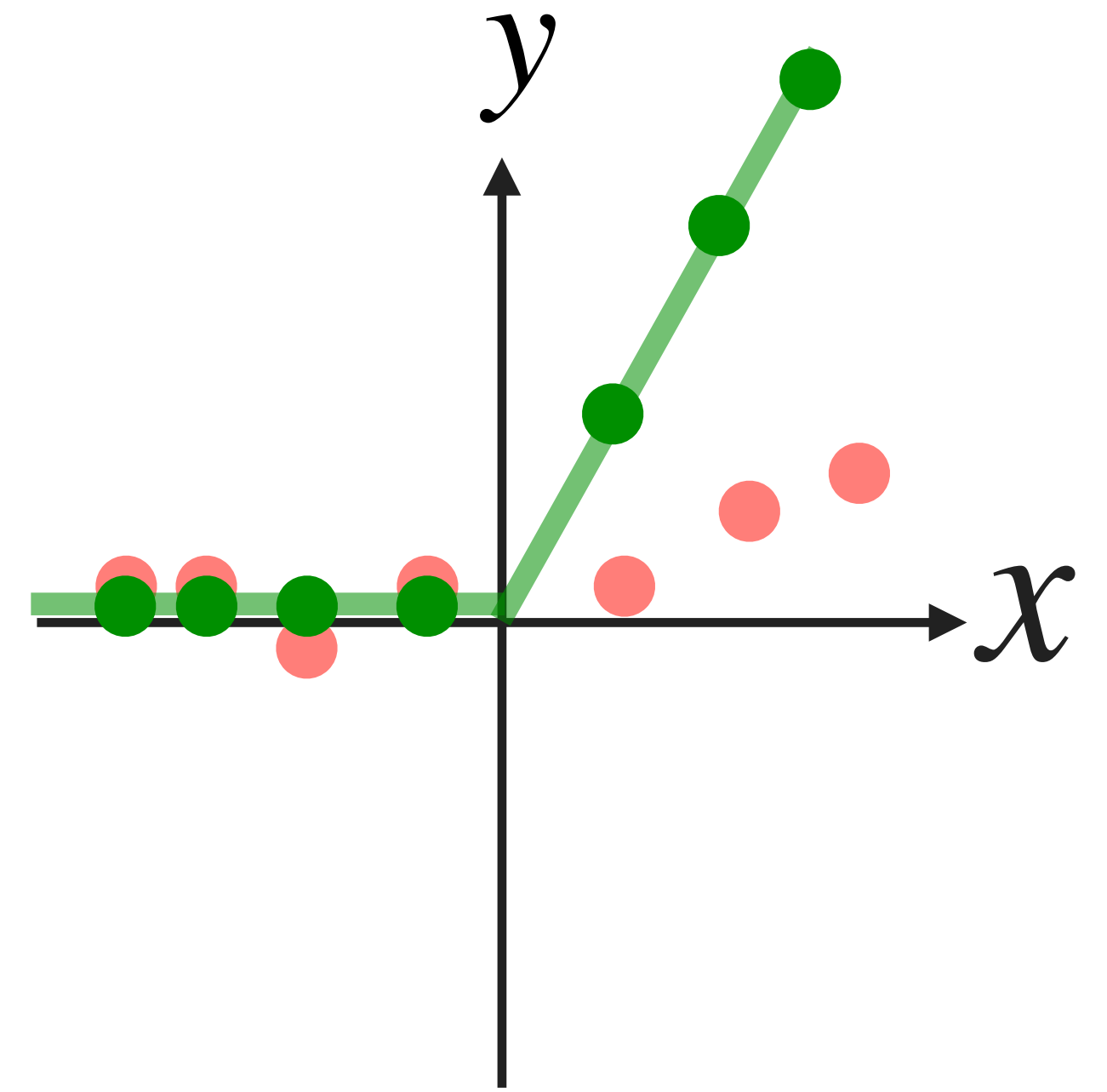
We first define a measure of error between the observed data and the model predictions:

$$\text{Data} = \{(x_i, y_i)\}_{i=1}^n$$

$$\text{Pred} = \{(x_i, \hat{y}_i)\}_{i=1}^n, \quad \text{where } \hat{y}_i = f_w(x_i)$$

We call this measure of error the loss function:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Introduction: Choosing w by minimizing the loss

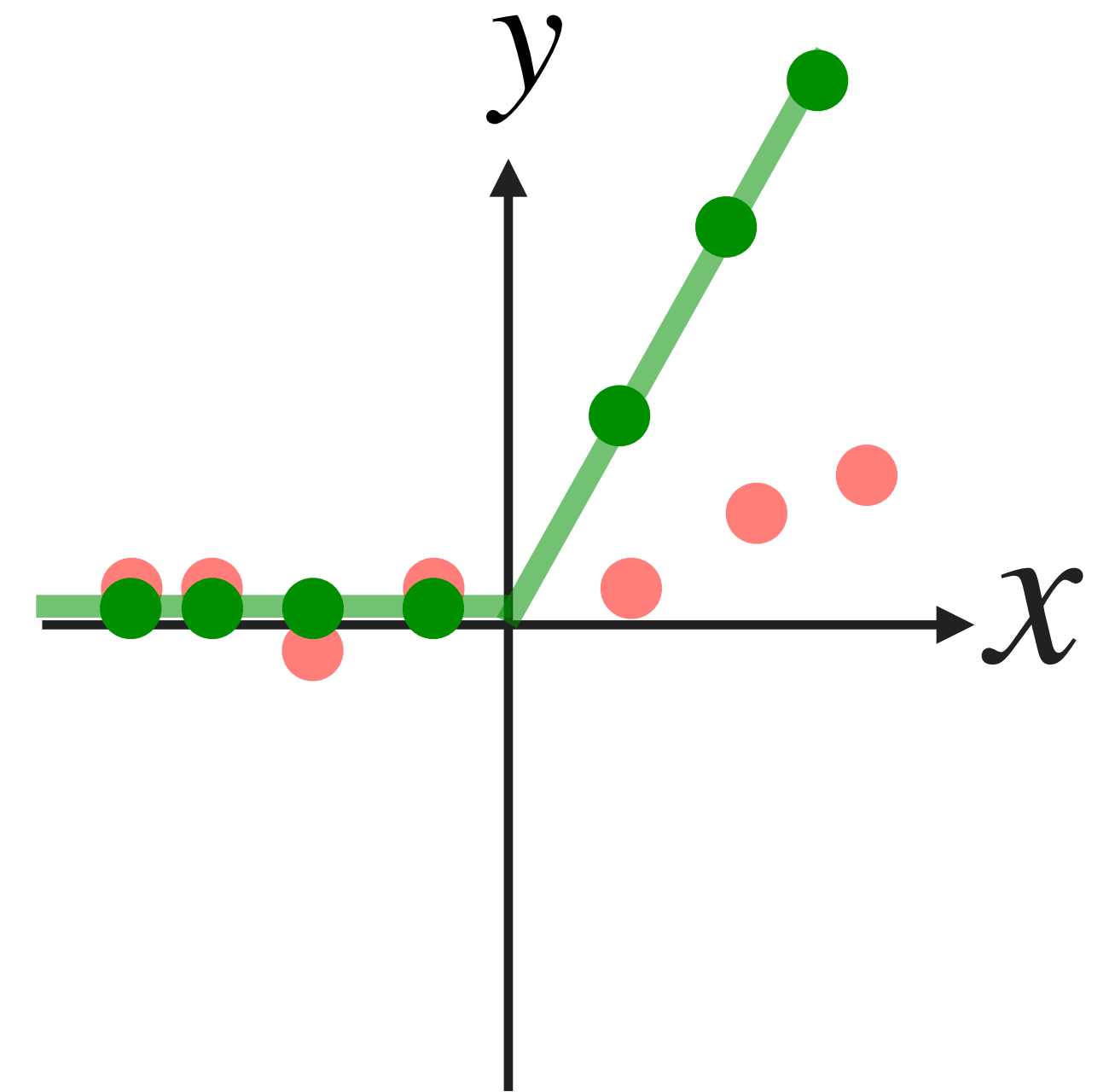
Now that we have a loss function

$$L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

we can define the best parameter as:

$$w^* = \arg \min_w L(w)$$

It means: Find the value of w that makes the prediction error as small as possible.



$$\text{Data} = \{(x_i, y_i)\}_{i=1}^n$$

$$\text{Pred} = \{(x_i, \hat{y}_i)\}_{i=1}^n,$$

where $\hat{y}_i = f_w(x_i)$

Introduction: finding w^* by Brute-force search

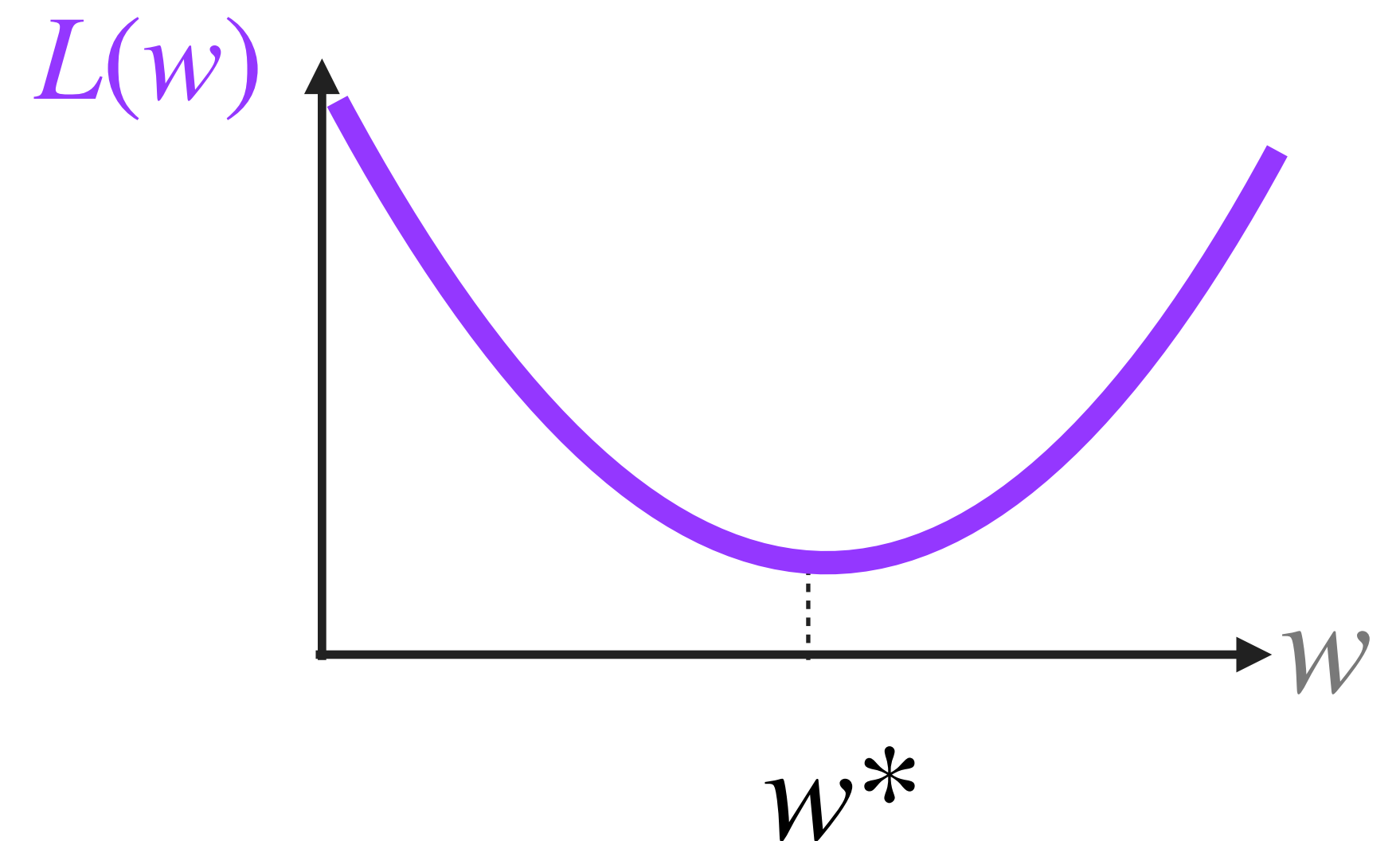
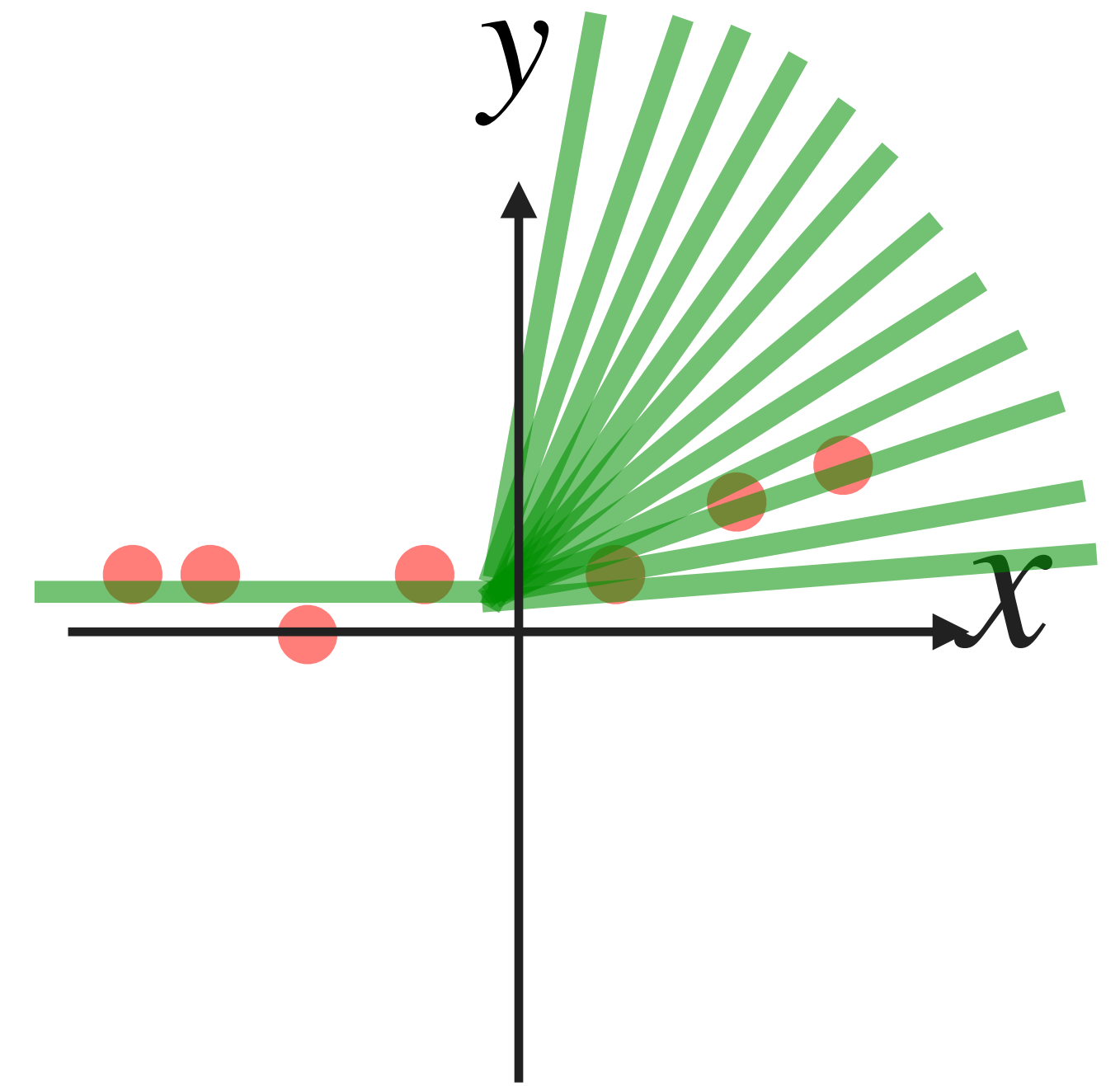
$$f_w(x) = \begin{cases} 0 & \text{if } wx < 0 \\ wx & \text{if } wx \geq 0 \end{cases}$$

One simple strategy to find w^* is:

Strategy 1: Try many possible values of w , compute the loss for each one, and choose the best:

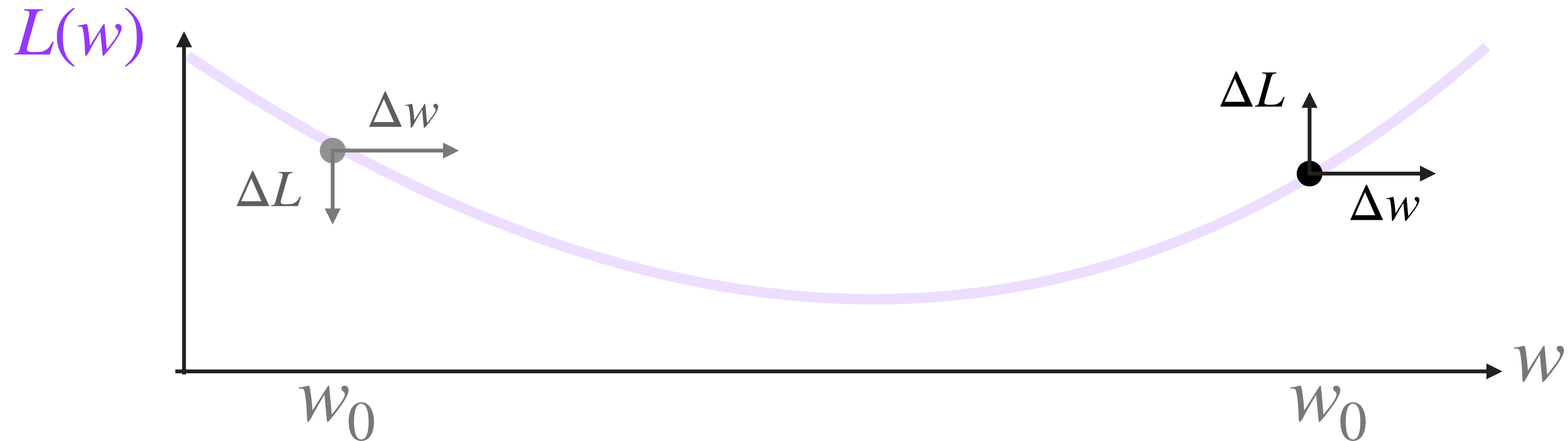
- Pick a candidate value w .
- Predict all outputs $\hat{y}_i = f_w(x_i)$
- Compute the loss $L(w)$
- Repeat for many values of w .
- Choose the parameter with the smallest loss, w^* .

But this strategy is computationally expensive, imagine when you have 10,000,000 parameter....



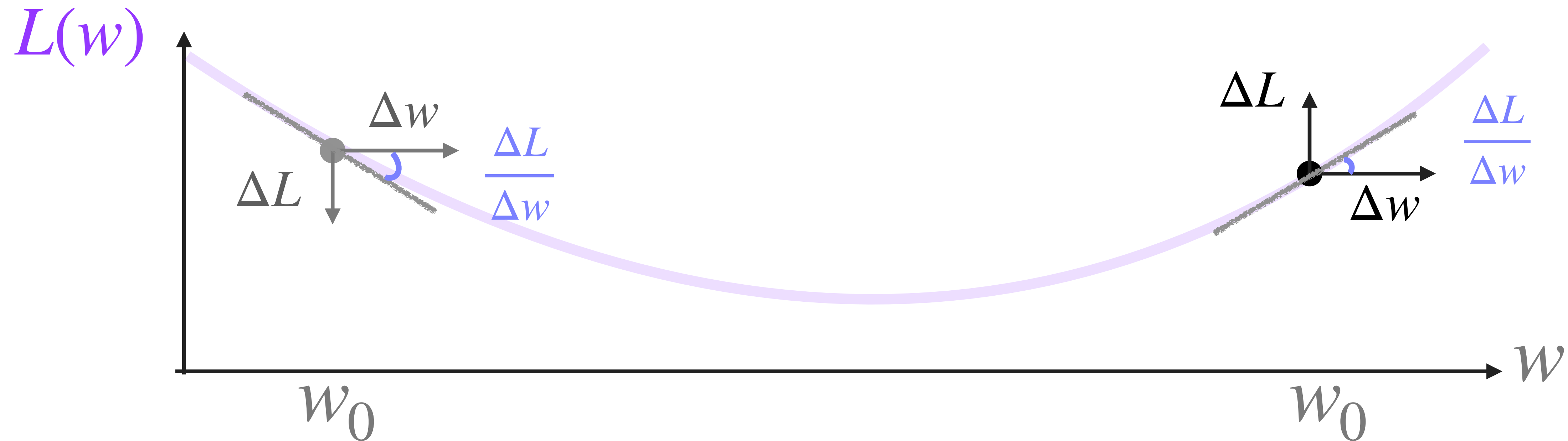
Strategy 2: Improving a guess (**gradient descent**)

- A. We start from a w_0 , calculate $\hat{y} = f_{w_0}(x)$ and $L(w_0)$
- B. Try a small change of w towards the positive direction: $w_0 \rightarrow w_0 + \Delta w$
- C. Measure what happens to the loss: $\Delta L = L(w_0 + \Delta w) - L(w_0)$



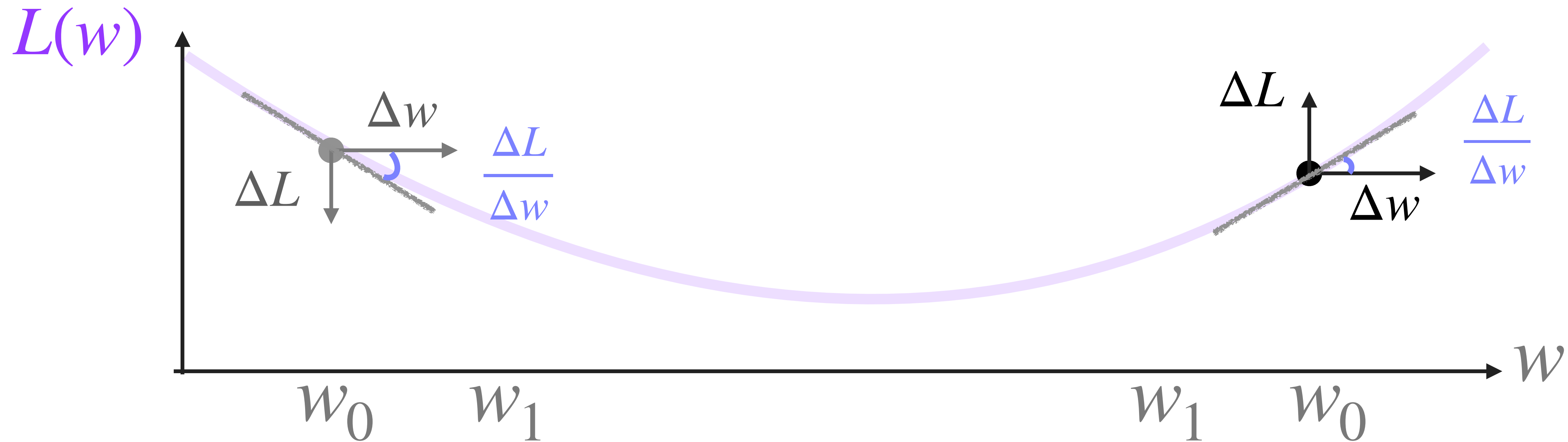
Strategy 2: Improving a guess (**gradient descent**)

- A. We start from a w_0 , calculate $\hat{y} = f_{w_0}(x)$ and $L(w_0)$
- B. Try a small change of w towards the positive direction: $w_0 \rightarrow w_0 + \Delta w$
- C. Measure what happens to the loss: $\Delta L = L(w_0 + \Delta w) - L(w_0)$



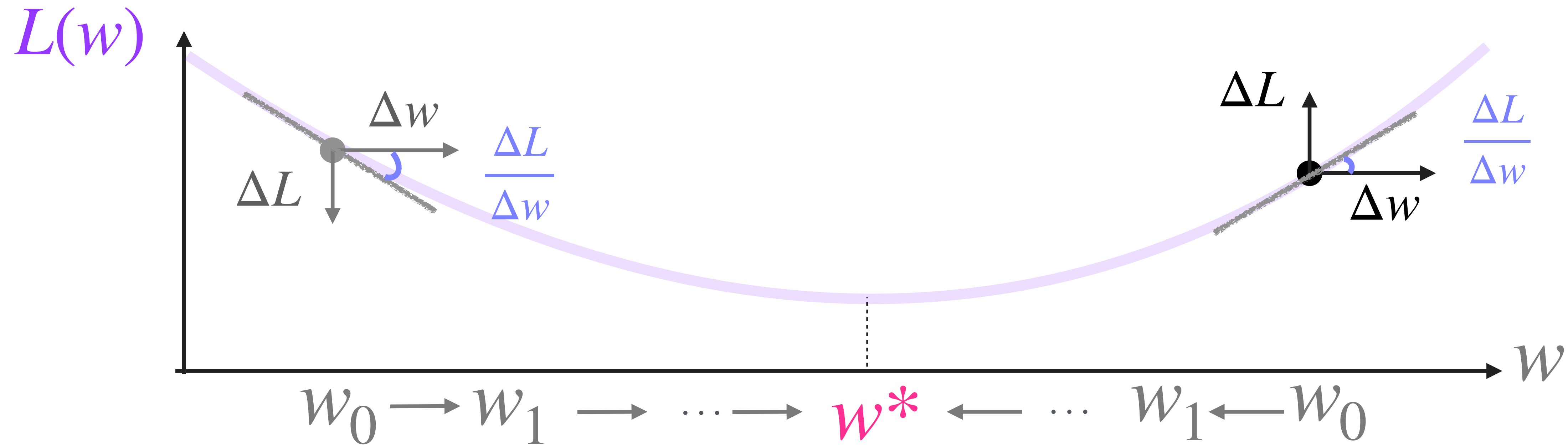
Strategy 2: Improving a guess (**gradient descent**)

- A. We start from a w_0 , calculate $\hat{y} = f_{w_0}(x)$ and $L(w_0)$
- B. Try a small change of w towards the positive direction: $w_0 \rightarrow w_0 + \Delta w$
- C. Measure what happens to the loss: $\Delta L = L(w_0 + \Delta w) - L(w_0)$
- D. Estimate the local slope of the loss: $\frac{\Delta L}{\Delta w}$
- E. If the slope is positive, move left. If the slope is negative, move right: $w_1 = w_0 - \eta \frac{dL}{dw}$ (η is the learning rate)



Strategy 2: Improving a guess (**gradient descent**)

- A. We start from a w_0 , calculate $\hat{y} = f_{w_0}(x)$ and $L(w_0)$
- B. Try a small change of w towards the positive direction: $w_0 \rightarrow w_0 + \Delta w$
- C. Measure what happens to the loss: $\Delta L = L(w_0 + \Delta w) - L(w_0)$
- D. Estimate the local slope of the loss: $\frac{\Delta L}{\Delta w} \approx \frac{dL}{dw}$
- E. If the slope is positive, move left. If the slope is negative, move right: $w_1 = w_0 - \eta \frac{dL}{dw}$ (η is the learning rate)
- F. Repeat until the loss stops decreasing

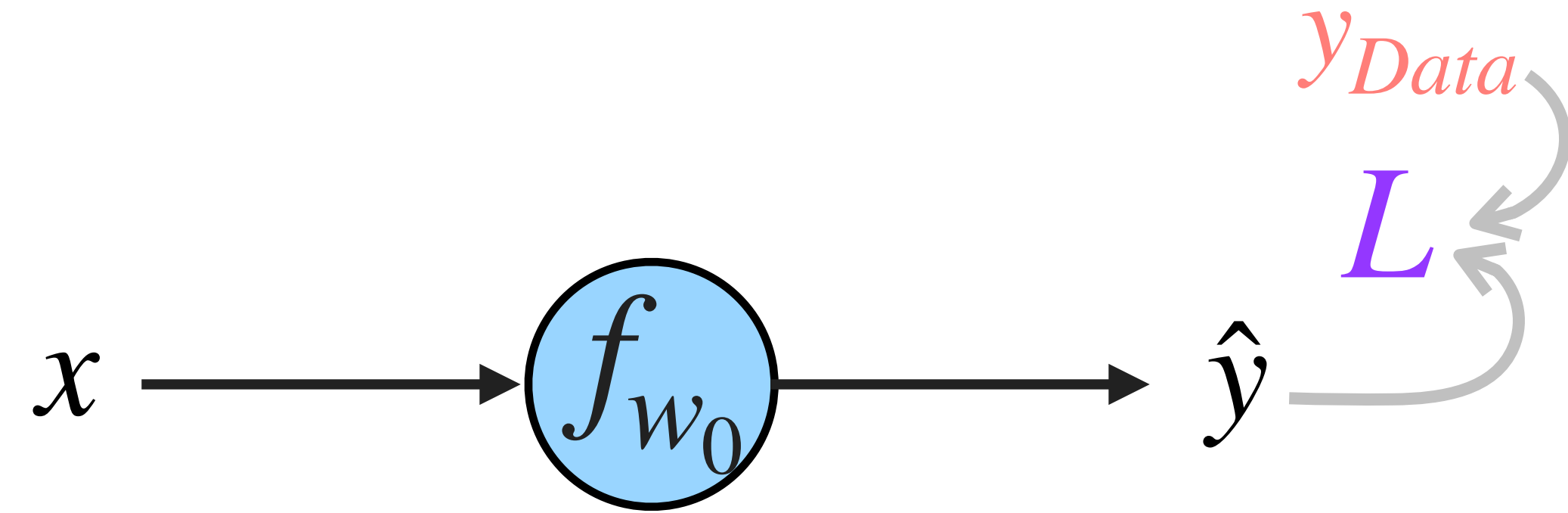


Computational graph of gradient descent algorithm

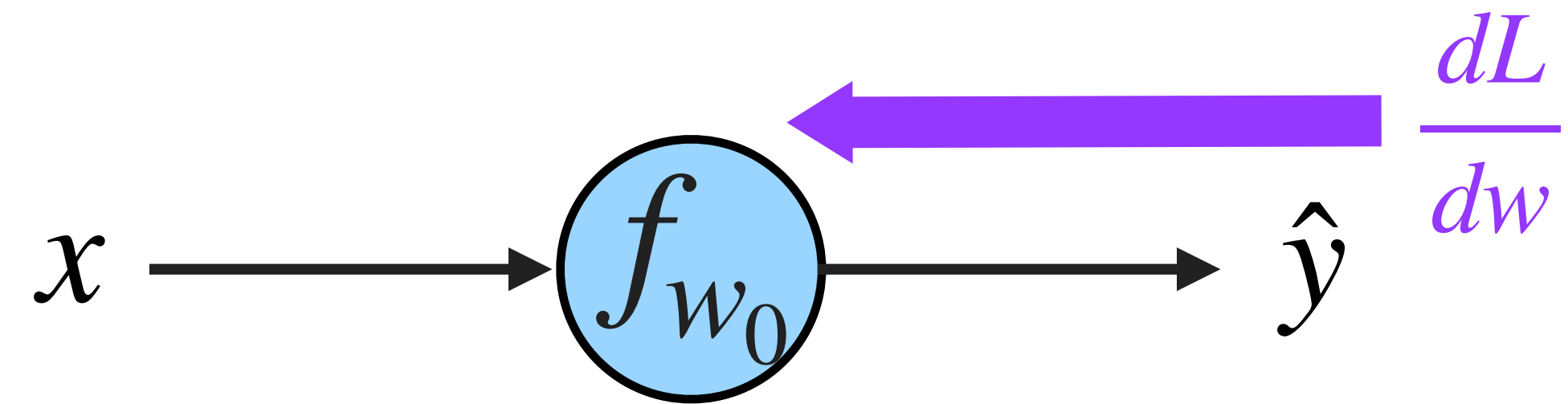
Strategy 2: Gradient descent algorithm:

Step1, Forward: Start from w_0 , calculate

$$\hat{y} = f_{w_0}(x) \text{ and from that, } L(w_0)$$

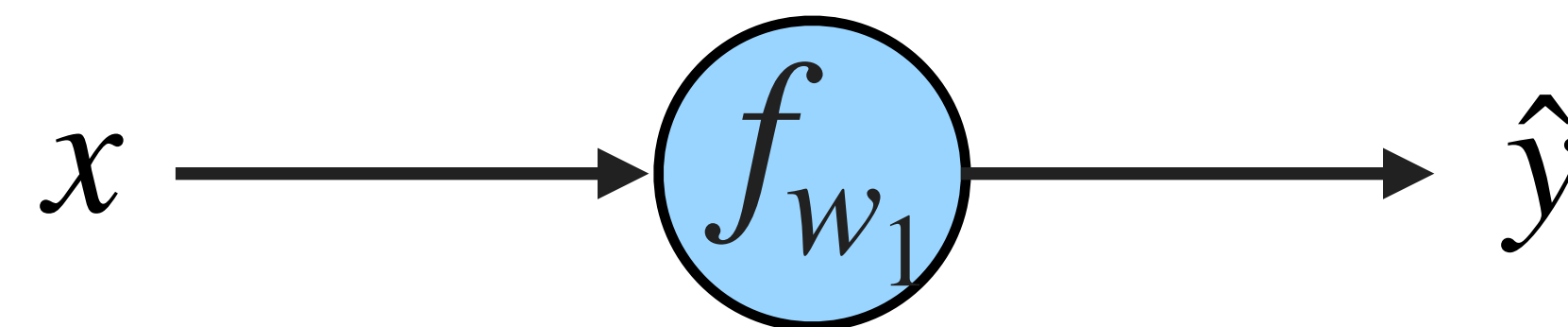


Step2, Backward: Calculate $\frac{dL}{dw}$ at $w = w_0$



Step3, Update: The new updated weight is

$$w_1 = w_0 - \eta \frac{dL}{dw} \quad (\eta \text{ is the learning rate})$$



Step4: Repeat until the loss stops decreasing



Introduction: What if we have multiple functions?

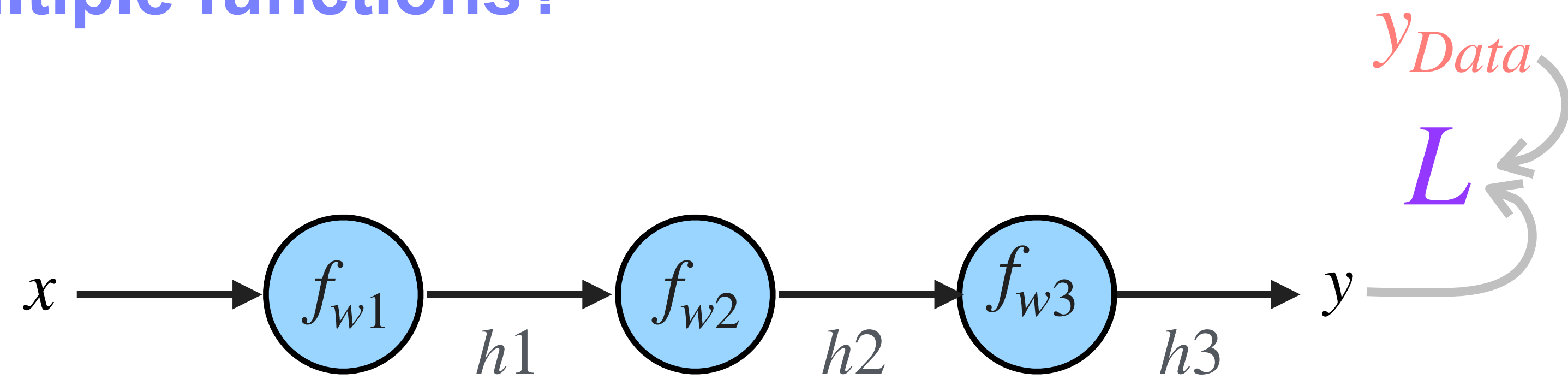
Step1, Forward:

$$h_1 = f_{w1}(x)$$

$$h_2 = f_{w2}(h_1)$$

$$h_3 = f_{w3}(h_2) = \hat{y}$$

$$L = L(y, \hat{y})$$



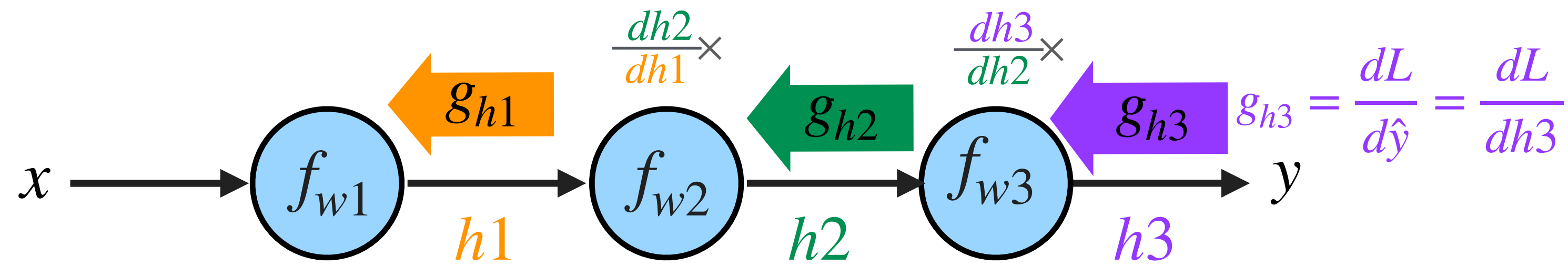
Step2, Backward: Let g be the backward gradient signal.

At the output $g_{h3} \equiv \frac{dL}{dh3}$

$$\frac{dL}{dw3} = g_{h3} \frac{dh3}{dw3} \quad \text{and} \quad g_{h2} = g_{h3} \frac{dh3}{dh2}$$

$$\frac{dL}{dw2} = g_{h2} \frac{dh2}{dw2} \quad \text{and} \quad g_{h1} = g_{h2} \frac{dh2}{dh1}$$

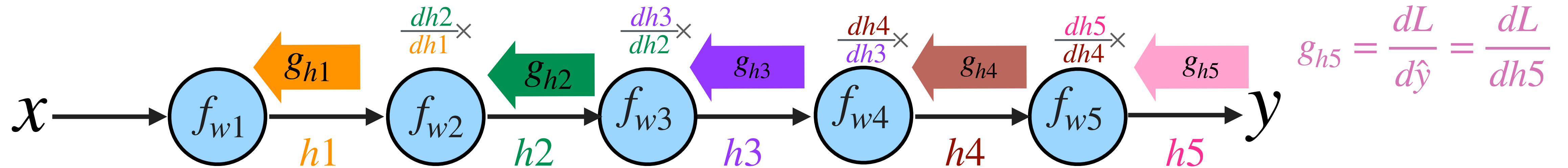
$$\frac{dL}{dw1} = g_{h1} \frac{dh1}{dw1}$$



Step3, and **Step 4** are the same as before

Introduction: **What if we have multiple functions?**

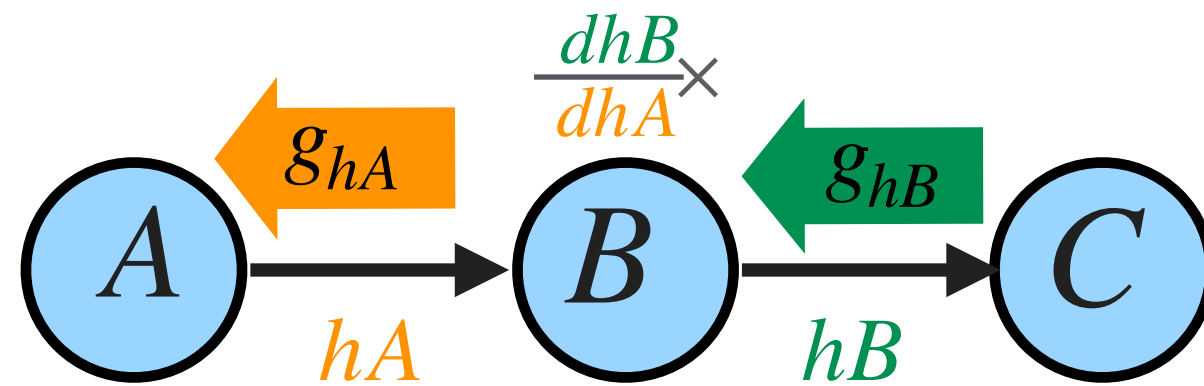
This pattern can go as deep as you want, and we just need to compute each local gradient g only once



Bonus:

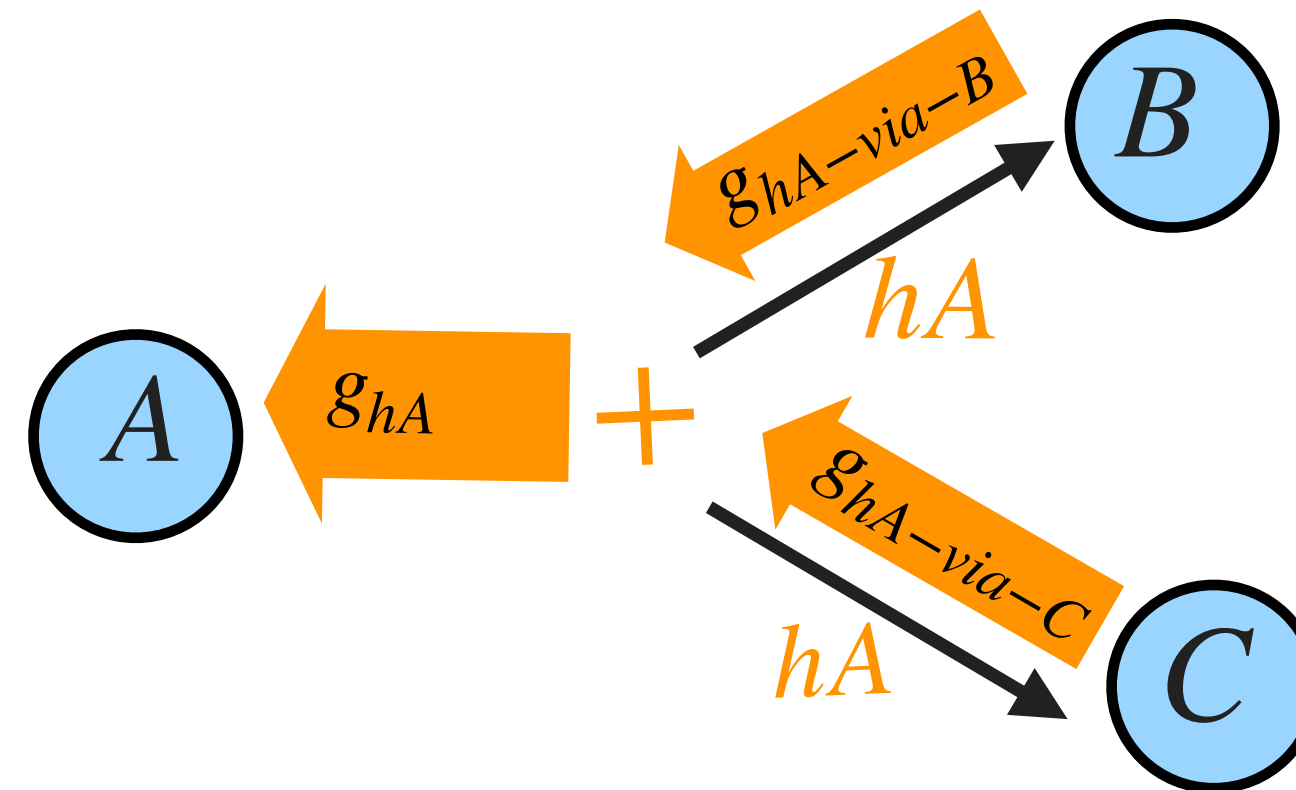
Introduction: What if we have multiple functions?

Sequential computation (chain rule)



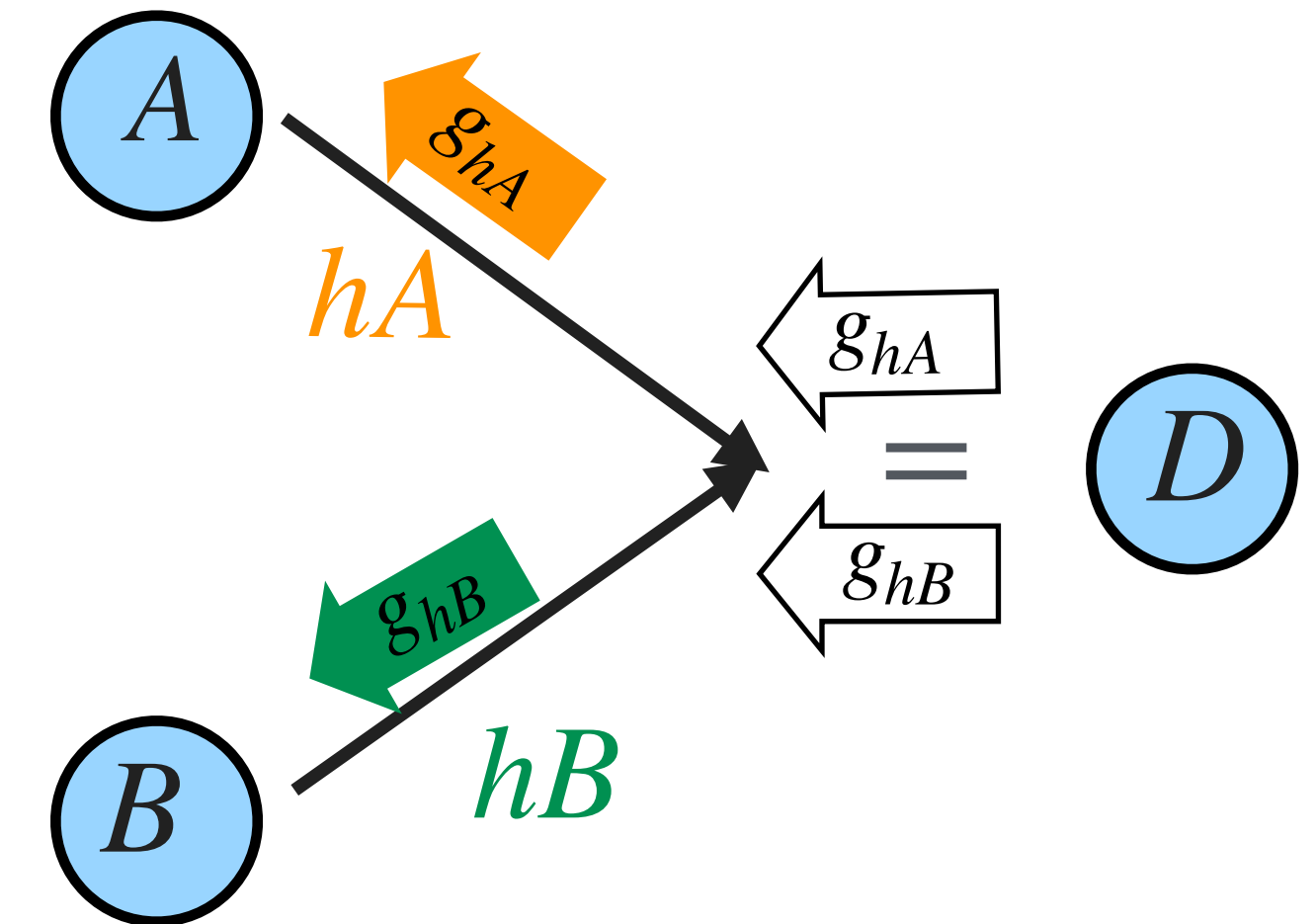
Gradient:
multiply the gradient by the
local derivative.

One value feeds multiple functions (split)



Gradient:
gradients from all outgoing branches are added at the split point.

Multiple values feed one function (join)



Gradient:

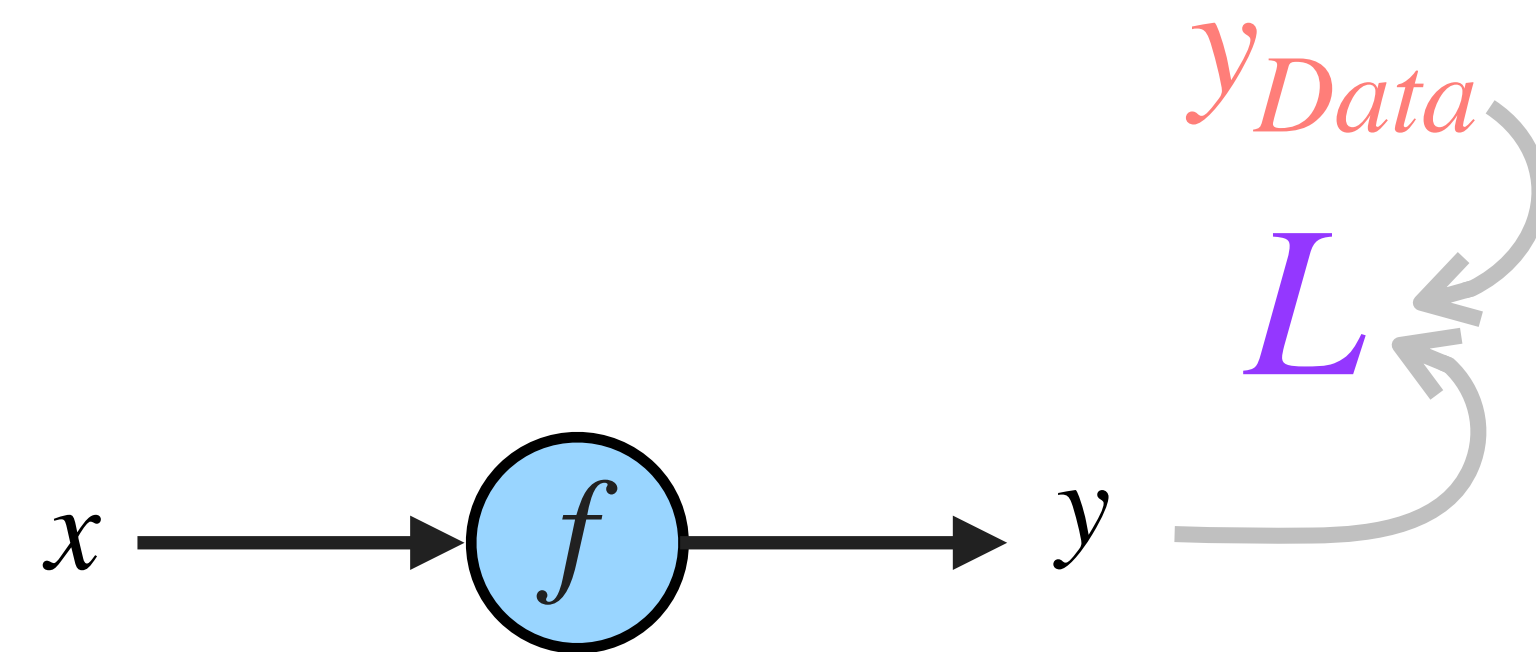
Each input receives a copy gradient from the source.

Closing: What we learned

We saw four pieces:

1. A model is a computational graph.

$$y = f(x)$$



2. A loss measures prediction error.

$$L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3. Gradient descent updates function parameters.

$$w = w - \eta \frac{dL}{dw}$$

Iteratively update w :

forward pass $\rightarrow$ loss $\rightarrow$ backward pass $\rightarrow$ update $\rightarrow$ repeat

4. Backpropagation computes gradients efficiently, when we have connected functions

